Journal of Object Technology Submission JOT-2013-014: Cover Letter

We thank the reviewers for their valuable feedback on this paper. As requested by the associate editor, the full text of each review is included below, along with our response to each point. Our responses to particular issues and questions is set in bold.

We revisited our submission to address the issues highlighted in the reviews. To better support the presentation of our research results, we included some more figures and sections, re-organized the section order, and extended explanations in several sections. In the following we address specific questions of the reviewers.

Reviewer 1

The paper presents a new design of an object-oriented language that extends Ruby with constraints. Constraints make give more freedom to express and enforce properties (e.g. assertions) that an application should satisfy.

Compared to systems based on functional and logic constraints, important expressive power here is re-establishment of constraints in case of changes, which is useful or many applications, including user interface layout. The edit constraints that directly support re-establishing a constraint in case of a localized change are useful, and give notable performance gain.

Another important aspect is soft constraints; these are helpful in practice because there is in general no guarantee that constraints would be satisfiable in a hard sense.

Being able to indicate when a constraint should hold within a piece of code is useful.

Some of these features are present in earlier systems such as Kaleidoscope, but the current design appears practical and is built on a reasonably high-performance infrastructure PyPy (as far as dynamic languages are concerned).

The system has 1.5 slowdown even if constraints are not used. Could this be eliminated using an extension of PyPy's type inference to detect variables that are not involved in constraints?

It cannot be determined statically which variables are not involved in constraints. However, the JIT learns which variables are constrained and which are ordinary object-oriented variables. In compiled machine code, the check is removed where possible (in the absence of constraints, the generated assembler is exactly the same as for the unmodified VM). In the presented benchmarks, any overhead is in code that the JIT has not compiled to machine code. The maximum slowdown of 1.5 is for the *binary trees* benchmark, where little code is compiled. We have clarified this in the section describing the benchmark results.

The system has not been evaluated on solvers that find entire data structures such as arrays or strings. The constraint solving is only applied to numerical domains in

the two example benchmarks presented.

We do have examples in the paper of solving for entire data structures – in the current system, though, these are interesting from the point of view of constraint expressiveness but not for timing comparisons, since the constraint satisfaction time for these is dominated by other factors. One such example involves solving for strings, namely the file contents for the preferences in the video encoding benchmark. However, the solution time for this benchmark is dominated by the video coding itself. We also provide a solver on arrays with numeric elements, that can express constraints over the elements, as well as the size of the array. We have included this in the section "Adding New Solvers". Finally, in the appendix, the electrical circuits example uses constraints on arrays to constrain the currents in a list of connected leads to sum to 0. We noted this in the explanation, and have also added a line reference. Since the DeltaBlue solver can operate on arbitrary objects, it would be easy to come up with additional benchmarks that use it – but for all of these we expect the constraint satisfaction time for these to be dominated by other parts of the program, since local propagation is so fast. We have yet to find good benchmarks that involve more sophisticated constraints on entire data structures; this is left for future work.

Assessment and suggestions:

I like this paper and the work done, but have a few concerns that justify a revision request.

First, there are only two constraint solving benchmarks presented, and that the presentation and the design mix the programming model and its implementation. It is my opinion that one of the main factors holding back progress in constraint solving support in languages is the lack of good challenge benchmarks, so the paper could do more to contribute to this.

As stated above, we have yet to find enough good benchmarks. Most related work provides benchmarks for finding solutions to puzzles, or finding data structures, which is not the main focus of object-constraint programming. Interactive layouting is a more realistic example of how we propose using object-constraint programming, which is why we provide that benchmark. As we develop more applications using object-constraint programming, we will be able to tell what a "good" benchmark is for this paradigm.

Second, the paper mixes the presentation of the programming model and the implementation. In a clean language, the programmer should not need to know how the feature is implemented. Currently, the presentation up and not

including section 4 is implementation-independent, but it does not cover all features. Some more interesting and novel features, like edit constraints, are explained essentially as implementation concepts. Is there a friendlier way for programmers to use edit constraints? Overall, I would have spent a little more on the description of the programming model than on some of the implementation details.

We have restructured the paper. We now present the theory of hard and soft constraints that we use in this work, then introduce our semantic changes first in an implementation independent manner and then give examples, and finally, we describe only what we consider the most important parts of the implementation. We have also made clearer which features we consider part of the language, and which features we consider are solver specific.

Third, I would have also liked to see a performance comparison with other recent systems, such as Kaplan system presented in POPL 2012 paper "Constraints as Control" and available (for an older version of Scala) here:

http://lara.epfl.ch/w/kaplan

The performance comparison is interesting because Kaplan uses Scala, which is statically typed, and interoperates with Z3 reasonably efficiently.

We ran three puzzle solving benchmarks to compare our performance with Kaplan, the Turtle language, and Prolog. The *Send-More-Money* puzzle was adapted from the Kaplan distribution, the *Layouting* benchmark from the Turtle distribution, and the *Animals* puzzle from a Prolog example. We're faster than Kaplan in those, generally slower than Prolog (which isn't very surprising, given that the benchmarks use no imperative features, and are really better suited for Prolog.) However, we have found that different versions of the Z3 library can vary wildly in their performance, and sometimes just changing the order in which constraints are declared can have a large impact. We have tried to change the order for each language to exhibit its best performance.

Fourth, the authors gloss over the questions of finite precision of numbers used by some solvers. Z3 does not support floating points, only real numbers, whose semantics is very different than floating points. SMT solvers that support floating points are only beginning to emerge. I wonder how the system enforces equality constraints on floating point values, given round-off errors. Does it assume some tolerance? If so, how does it ensure that the tolerance is well-chosen for the particular domain?

This was initially omitted, because the Z3 C-API returns results as double precision floating point numbers. We now note this issue for Z3. Currently, programmers have to be aware that there may be precision loss here. Cassowary uses floats internally, and uses a hard-coded ε to determine if two floats should be considered equal. We have not, in practice, had problems with this, but we agree that the question remains how much the programmer has to be aware of precision issues. We will investigate this further, but conclude for now that the programmer has to be aware that rounding issues may occur when using floats. Z3 also supports integers, and Babelsberg/R represents Ruby Fixnums as Z3 integers if possible.

Fifth, as far as I can see, the presented system does not support enumeration of solutions. It is possible to find all solutions when the set is finite?

Indeed, enumerating solutions is not currently available. We have made it more explicit in the comparison with BackTalk, Ilog, and Kaplan in the Related Work section, as well as in the theoretical background section, that this is not among the primary goals of Babelsberg. We think that enumerating solutions will be useful for debugging and plan to implement it primarily for that use case. We also plan to provide Kodkod as a finite domain solver, and enumerating solutions will be useful here as well. For now, however, in Babelsberg we are interested in *a* solution, so if there are multiple, the solvers are expected to select one arbitrarily (and we state this now in the text).

I was also wondering if identity constraints are useful for finding data structures?

If we supported enumerating solutions, possibly, yes. This would be similar to BackTalk or Squander, where a constraint can be satisfied by searching for appropriate objects on the heap or creating objects and testing them for suitability. In our case, it is useful for modeling data structures where the difference between "we need two values to be equal" and "two values need to refer to the *same* object" is important. This is an issue, for example, in a Doubly Linked List, where `self.previous.next` should be *identical* to `self`, not just equal. We have added a reference to the Kaleidoscope paper on object identity and provide a short example.

Could the authors provide a reference to the DeltaBlue library and examples where they used it?

We now introduce solvers that are important for our discussion in the introduction, among them DeltaBlue, and have included a reference. We have also added an example of using it in the section ``Solving Constraints''.

Reviewer 2

Summary
=======

The paper describes the language Babelsberg, and its implementation. Babelsberge is Ruby, extended with primitives for expressing constraints between program variables. Babelsberg serves as an interface between an OO language and constraint solvers: different constraint sovers can be plugged in to solve particular constraints.

Evaluation
==========

As the athors do, I too believe there's value in integrating OO widely used OO language and constraint solving, so the authors are working towards a worhty contribution. This paper is an attempt to explain that contribution. It is, however, inadequate.

The biggest problem with the paper is that the language is _only_ explained through examples. The semantics, not even syntax, of the extensions are explained in any general level, so it is really not possible to know what Babelsberg is. There is a section on implementation, but that, on the other hand, is too detailed, and vague at the same time. I don't believe that it is possible to infer the semantics, or even syntax, from that description either.

The syntactic extension is now explained as being optional at the beginning of the design section and in the overview. The syntax of the always construct is the same as for a Ruby method call. We now refer to a description in the appendix.

We have re-structured the paper extensively to introduce the constraint theory first, then explain our two semantic extensions in an implementation independent way, followed by an example. The implementation section has been shortened to focus on what we consider the important details.

In general, the writing is not precise and unambiguous enough. Too many details are crammed into the paper, leaving many (most) unsatisfactorily explained. Terms are used without explanation, new ones invented, etc. I give many detailed examples below on such problems, and how to possibliy improve. Please note that going through the list of my recommendations below and following them would not (yet) make this paper accetable.

Detailed comments
==========

Pg 1

"standard imperative programming"
- I'm not sure what 'standard' means here

We've removed the word "standard", and replaced this phrase with "We present our design for integrating constraints with a dynamic, object-oriented language that preserves a familiar imperative programming model". We have also discussed prototypes and our JavaScript-based implementation, to emphasize that this design is not bound to Ruby-style classes.

Pg 3

"... Babelsberg is a direct extension of a standard object- oriented model (with dynamic typing, object encapsulation, and standard classes, instances, methods, and message sends)."
- I would avoid the use of the word "standard" here as well.
Babelsberg seems to be quite tightly bound to Ruby and its take on OO concepts.
I'd especaily not say "standard classes": I don't think such a thing as a standard, even de-facto, for OO classes exists.

We've removed the word "standard" and made mention of prototypes and our JavaScript-based implemententation, to emphasize that this design is not bound to Ruby-style classes at all.

The second to last bullet repeats almost verbatim on page 11.
Is that intentional?

Yes, this is intentional. The paragraph has moved, but the repetition is still there. The reason is that we think that the distance between the two sections is large enough to warrant a clear reminder. We consider this preferable over paraphrasing, so the reader can clearly connect the section with the contribution in this case.

Pg 4

"Programs frequently have some set of constraints that should hold."
- This begs for a sentence that gives an example.

We initially added examples, but due to restructuring, this sentence is now gone

The first code example is rather poorly explained.
- Please, in the text, point to what the constraints are in the code.
  Like this: "We want to make sure that ... is never less than 100 and that...
  These condiditions are expressed with the /constraint/ on line N" or something similar.

We have added line references and hope we explain each code line more clearly now.

- The caveats about why some constraints are omitted are confusing. Especially the
  reason that "conjuctive constraints" are already demonstrated is confusing, because
  you have not explained what those are. (You have not yet really explained constraints).
  Maybe just include all constraints that arise from your verbal explanation. It's
  just one line of code (though I don't know what "corner" is in your code).

We have rephrased this paragraph to explain the conjunction. We hope it is clearer now that the additional tests are simply omitted for simplicity.

"Imperatively, we can use ... aspects".
- I don't know what "imperatively" has to do with aspects. Perhaps, "In Ruby, and in other
  languages that support aspects, ..."
- For those readers that are not familiar with AOP, a few explanatory sentences might
  be in place.

We have rephrased this to make clear it is really up to the programmer to satisfy the constraints, and that AOP is just a possible solution. We include a reference for readers not familiar with AOP.

"The original constraints are expressed in a form that makes them harder to understand."
- What are the "original constraints"?
- "harder" than what?

We have rephrased this to elaborate that branches and assignments require programmers to carefully check and map the resulting control flow to the desired constraints.

Pg 5

There are actually ... -->There are ...

Done

"... is not represented explicitly, but instead is implicit..."
- Maybe it is enough to say it is not explict (the "but implicit" part seems redundant).

Done

"Since there isn't a declarative specification of what an optimal solution is, it is nontrivial to decide whether the solution is optimal."
- This seems like stating the obvious
- What is "the solution"?

Clarified. The questions are whether any solution is found, which one (if there are multiple), and which properties determine which solution is selected.

"In the presence of competing soft constraints, i.e., multiobjective optimizations, this becomes even harder."
- You have not defined what "soft constraints" are.

We have added a forward reference to the in-depth theoretical discussion.

- You seem to say that "competing soft constraints == multiobjective optimizations". I guess yes, but please define/explain these terms.
- What does "this" refer to? Deciding whether a solution is optimal?
The goal as I understand is finding an optimal solution (BTW, explain what you mean by optimal), not deciding whether some solution is optimal.

The goal of the code is to find any solution, but the users of the code may want to know whether the solution is optimal for their purposes.

"Thus, it is usually clearer to express and satisfy the constraints explicitly."
- Thus?
- clearer than what?
- the? (constraints)
- What does "satisfy explicitly" mean?

The introduction was changed to clarify that this is related work (Reviewer #3 also found this confusing and considered the code to be Babelsberg code) and that libraries address some of the problems of the imperative solution

"the imperatice code" --> "imperative code" (?)

Done

type domain should probably be in italics

Done

"whether the solver can use the constraints in a general way (e.g.,
for c = a + b, to solve for any of a, b, or c; or as a dataflow
constraint, i.e., to only use the constraint to find a value for c);"
- This is a really fuzzy way of explaining that some solvers can
solve equations (or other relations), while others can only select and
evaluate one of a given set of functions.

This was changed to state this more explicitly

"and whether the solver supports soft constraints as well as hard (required) ones."
- Explaining soft and hard constraints like this as a side-note is not sufficient, I think.

With the previous forward reference and the immediately after the Related Work following section on soft and hard constraint theory, we think this is

of particular note --> of particular interest (?)

Changed

"previous, purely imperative"
- you should really explain what you mean by "imperative". Is it
non-declarative? If so, the AOP solution surely is not purely imperative. Is it the opposite of "pure functional" as in no side-effects?

This was changed to say "previous solution that relied on state changing operations and branches"

If so, that doesn't seem to fit all that well either.

"# boilerplate code as for purely imperative approach"
- I don't understand this comment? Do you mean that you have
omitted some code that would make this example complete?

We now include the complete code

"For specialized application domains such as user interface layout,
constraints are sometimes available via a separate dsl that describes
relations between visible objects that can be automatically maintained
by the runtime."
- This sentence keeps going a bit too long, and the thought is lost.

We have split the sentence and clarified.

"This approach allows programmers to specify constraints and avoid boilerplate code to trigger constraint solving, and has found widespread adoption and renewed interest recently in particular through the Mac OS X layout system."
- This approach -->This DLS (?)

Changed

- and avoid boilerplate code to trigger -->
instead of writing boilerplate code for triggering

Changed to "without having to write boilerplate code for triggering"

- , and has found... -->Such DSLs have recently found widespread adoption, and have renewed interest toward constraitns, in particular ...

Split into separate sentence and clarified

"Some languages have built-in support for data flow, which allows ..."
-->
"... for data-flow constraints: programmers can ..." (?)

Not changed after consulting with other native speakers.

First you say:
"allows programmers to express unidirectional constraints between objects and their parts."
then:
"Programmers can thus directly express relations between objects."
- aren't these contradictory?

Unidirectional relations, really. We removed the second sentence, it was not really necessary

and returns the --> and it returns the

Not changed after consulting with other native speakers.

"Another approach to supporting constraints — and the one adopted in the work reported here — is to integrate this support with the programming language itself."
- This is a start of a new section, so what "this support" refers to is not clear.

Changed to "is to integrate constraint support"

- the programming language itself -->a general purpose programming language.(?)

Changed

"One of the most widely known of these approaches evolved from logic programming, for example the Constraint Logic Programming scheme [19] and instances of this scheme such as CLP(R) [20] (which provides

constraints over real numbers)."
- This is not a sentence. Chop and clarify.

Changed

Pg 7
---

at the same time sacrifice --> at the same time they sacrifice

"sacrifice familiar capabilities and programming style. "
- Familiar to OO programmers perhaps. Maybe add some words along
the lines: "OO is the mainstream programming paradigm, and to bring constraints to the large OO community , our goal is to ..."

Changed to "but they sacrifice the familiar capabilities and programming style of the more mainstream object-oriented paradigm"

"Kaleidoscope supported standard classes"
- again, what are standard classes?

Changed to "Smalltalk-style classes"

- supported --> supports (the system exists still? maybe remove past tense from the entire paragraph?)

Unfortunately, the authors have no copy of the system anymore, so only the description remains.

To support this, --> Concretely,

Removed during the rewrite of this section

that then expanded this --> that expands

As above, we think the past tense is appropriate

"the language also provided methods"
- Does this mean OO methods or constraint satisfaction methods? Please clarify.

Clarified, it means OO methods.

Pg 8
---

features of the language --> features of the Babelsberg language

This paragraph moved to the introduction and the sentence was changed.

Why in the example are both farenheit and celcius initialized?
Would the constraint not take care of initializing one from the other?

The constraint will initialize the correct values, but to declare types, we have to assign a number. Changed and an explanation added.

Pg 9

"Here these are just accessors"
- Here seems to refer to "constraints" but that doesn't seem to make sense. Constraints are not accessors?

"Here" was supposed to refer to the example, "these" to the methods "centigrade" and "fahrenheit", but the sentence structure was misleading. We have clarified it.

"rectangle from Section 2" --> the Rect class shown in Figure ...
(please be precise with your references)

We did not want to label all listings, but also did not want labels on just some. We changed this to a page reference.

Pg 10

"repeatedly evaluating the block"
- what block?

Meaning the Ruby block passed to the method. Changed to read "constraint expression", to be more generic

- aside: the property you are describing is idempotency (?)

Yes.

be treat --> be to treat

have not supported --> do not support

"we believe that the additional complexity of keeping track of
successive versions of a variable outweighs the additional
expressiveness"
- you are saying "complexity outweights expressiveness" which, while I understand what you are getting at, sounds odd.

These three sentences were changed as suggested, but later removed during editing.

and so can appear --> and so they can appear

Not changed after consulting with other native speakers.

"just as with any other statement"
- maybe drop? you already listed all kinds of statements, and said "and so forth".

Done

"The sum method makes of the Inject"
- " .... makes use of the ..."
- please epxlain this injection feature (is it a Ruby-thing?)

We have extended the footnote. "inject" is a collection method known from Smalltalk. It is available in other languages as "reduce" or "fold".

Pg 11

First paragraph of 3.3. repeats from Intro.

As explained above, this is intentional.

What is the syntax for specifying which solver to use?
(In the case where types do not determine the solver unambiguously)>

It is simply an additional argument to the always method. We have added example code for this in the "Solving Constraints" section, and describe it in the appendix

repeatedly checked --> checked

Done

The reference to "co-operating constraint solvers" is confusing.
I, at first, thought that you are throwing a single constraint
to many solvers that somehow collaborate to find a solution;
but after reading the appendix, it seems that you just
assign a different solver to each constraint. I guess I am lost here,
as this is how I thought your system already works? Based on
a type of the variables, the constraint solver to apply is selected(?)

The system already supports different solvers in the same program and selects which solver to use based on the type of the variable. However, different solvers cannot collaborate to find a value for the *same* variable – each variable is always determined by one particular solver. The cooperating solvers architecture allows multiple solvers to collaboratively determine the value of the same variable. The reference has been removed since this is part of the future work and not yet implemented.

"upstream"
- you have not defined what this means. I know this is an established term
with data-flow constraint solvers, but OTOH the article is written for a more general audience, so
please define and/or explain from where the terms come from.

This was removed during editing.

"or imperatively" ???
- does this mean "assigning to the variable directly"(?)
- if so, would it not then violate its constraints?

We have added some explanation. The constraints are checked on assignment, and the "read-only" constraint is updated. A "read-only" annotation does not "freeze" a variable, it just prevents the solvers from changing the variable value to satisfy constraints. This is also explained in more detail in the section on relevant theory now.

Pg 12.

This applies to the reference to Section A.2. and to several other
references to appendix (or future technical report):
- I think the his is a bit unpleasent for the reader. Ideally, proofs,
longer code examples, and such are what would be found in a tech report.
Here, you offload discussion of ideads and concepts to the tech report,
which I think belong to the main paper.

We have moved relevant parts of the appendix into the main paper. Only a reference to the syntax of the "always" method to the appendix remains.

"associated block"
- please define

Again, this was Ruby terminology. Changed to "closure"

"If multiple variables that have constraints on them are assigned
using multi-assignment, all values are assigned before the solver is
triggered."

- Could this lead to conflicting (unsatisfiable) constraints if
those variables are involved with more than one constraint?

Yes. As with normal assignment, if as a result of these assignments the constraints cannot be satisfied, the assignments are not applied and an exception is raised (which the programmer can handle.) We have clarified this in the text. The reason for not applying the assignments in that case is, that the program may continue after an exception, in which case the system is still in a valid state.

Section 3.6:
- OK, here's the explanation on soft and hard constraints. It comes too late
(or at least add forward references to whenever you use these concepts).
Could you somehow explain what these are in a more general fashion, rather
than _only_ resorting to an example? The notions of
over and underconstrained systems might help in explaining.

This was shortened and the theory on soft constraints now appears prior to this.

You use priority "high". Does it have an exact meanign? (or any meaning?)

We have added a detailed explanation of the relevant constraint theory. The theory allows an arbitrary number of preferences, but prior work has found that only few priorities tend to be used in stylized ways (see "The Cassowary Linear Arithmetic Constraint Solving Algorithm" by Badros et al.) The symbolic names are just for clarity of exposition. There is no ordering between different "low"s in Babelsberg, so the solver is free to trade them off arbitrarily, as specified in the semantics of hard and soft constraints described in Section 3.

Pg 13.

"when combining constraints with imperative programming, since in such a language "
- imperative programming is not a language

Fixed

You epxlain stay constraints here, but I think you did so earlier in the paper already.

They were mentioned in passing in the Related Work section on Kaleidoscope. We now refer back to section 3.2

What does priority "low" mean?
(In relation to "high", and is there "must" or "even lower" or "middle"...,

and is there an ordering between the "low"s?)

See above explanation for question about "high"

In 3.7, the example you give about incremental constraints seems incomplete.
It is difficiult for me to see how the TemperatureConverter system could
benefit from any incremental solving? Can you provi de some more details?

The idea is that the input to the slider, instead of an enumerator, may be a graphical slider. We have added this to the text.

Or is the semantics of your "edit" language construct that it
invokes an incremental solver if one is available?
If so, how does a constraint specification declare that an incremental
solver can be used? Or does it have to? does the unerlying solver know?

We have added further explanation of the "edit" method. It is provided by the Cassowary and DeltaBlue libraries, so the programmer explicitly selects and invokes the incremental solver as required.

Pg 14

Are your "OO constraints" something built-in, or are they just part of
your "standard library" that comes with Babelsberg?

Assuming this refers to the constraints on identity, class, or interface, these are just examples of how ordinary reflective methods of the underlying OO language are supported in the design as constraints.

"The interpreter normally operates in imperative evaluation mode."
- meaning of imperative here is unclear. Maybe you are trying too hard
to make a "language-philosophical" point. In perhaps simpler terms,
you have embedded constraint specifications to Ruby, and your interpreter
is otherwise a standard Ruby interpreter, but it recognizes these constraint specificaitons and swithces to constraint solving mode wheneve it encounters one.

We have significantly shortened and simplified this section.

Pg 15:

5 -->five
- style guides say that small numbers (up to 20) should be written out

Done

re-assign a constant --> re-assign to a constant

Done

3 -->three

Done

, have --> , we have

Done

"object-oriented (OO) values"
- what are these? objects?

Changed. To distinguish them from objects used in the solvers (which are objects in the host language that are treated specially), we now call these "ordinary objects"

4.2
The section about implementation is not useful.

We have shortened the implementation section. Some of the details have been removed. Details which we consider crucial to understanding our implementation have been expanded and changed to be more comprehensible, and to focus on the relevant issues.

It assumes details that the readers will now know, will not be able
to follow. It essentially explains the code, but does not show code.
Readers will get very little out of this section.

Examples:
The text assumes that the reader knows some class Interpreter.
Where is this class defined? Part of the framework you are using?

The section dives into what happens with individual bytecode instructions,
with no general outline given first.
BTW, what is a "constraint value"?

These are objects used in the solvers. As with "OO values" above, this was changed to "solver objects" throughout the paper.

"Constraint Construction" subsection, first
paragraph, is a description of the control-flow, assignments,
method-calls (to methods not explained, but only mentioned in
passing), of a function or two. If all these details are necessary,
I'd much rather read the code; or pseudo-code.
In this form, I cannot really follow the presentation.

We have simplified this section.

Is a detailed implementation section really necessary at all?
Can you describe the one, two, three interesting/key points of the design/implementation, and leave the details out?

We have focused on the relevant issues, and removed less relevant details.

4.3
My comments to section 4.2. apply here too.
Please first give the full syntax of an edit constraint specification, then describe its semantics, and only then try to to explain the implementaiton (if you think it is necessary).

We have split the explanation of edit constraints into theory, design in Babelsberg, and the relevant implementation details in Babelsberg/R. We hope this clarifies that edit constraints have no special syntax, and are just methods defined by the libraries that support them.

Pg 19

"Object-oriented performance" is an odd term.

Changed to "Performance of Pure Object-oriented Code"

Toward answering -->To answer (?)

Done

Pg 20

"Considering that Babelsberg/R is aimed at imperative programmers, we expect that most of the time the vm will not be solving constraints in tight loops, but running mostly imperative code intermingled with constraint re-satisfaction."
- Are you sure about this assumption?
Why would the programmers not use the whole arsenal of features throughout?
In any case, I don't think such assumptions are needed to justify your system.
Just report what the performance is, and then users can decide where it can be best used. It is understandable that there is some overhead.

We removed this sentence. We did not mean to imply constraints would not be used. However, in the same way that expensive operations are generally avoided in tight loops in object-oriented programs (e.g., most call sites are mono-morphic, reflective features are rarely used – assumptions which are the reason why JITs work so well), we believe that object-constraint programs likely do something in addition to repeatedly solving constraints. In the video streaming example, besides solving constraints for every frame's quality, each frame is also encoded and written to a stream.

- And "mostly imperative code" --> "standard Ruby code" (?)

Yes, this would have been more correct.

THe difference between "Constraitns" and "Edit Cosntraints" is huge.
Can you offer an explanation? Is it that the former runs a "planning" algorithm
every time, but the latter re-uses the same plan, i.e., data-flow?

This difference is expected, and we hope sufficiently explained now with the theory on incremental solving.

less 1.5 --> less than 1.5

Done

due an --> due to an

Done

Pg 21

The details of the video application are kind of uninteresting.
One can always hide all differences by including computationally intensive tasks
to the measurements. How about instead reporting the wall-clock time of solving
the realworld application's constraints (video app), and then say that
of course, it is miniscule compared to the minutes it takes to encode/decode
the video. But it would be nice to know if the wall-clock time is 10ms, 1s, 5s?

We have shortened the description of the example, and report instead the time spend on encoding, I/O, and constraint solving separately for the object-oriented and object-constraint version.

Pg 22
---

was minimal --> was small

Done

"A MethodTimer can be instantiated with a class and the name of a method, and wraps..."
- some grammar issues here

Split into multiple sentences.

Pg 23
---

connecting variable --> connecting variables

Done

References
---

jit --> JIT

[4], number of pages missing? number missing? volume?

[7] Title seems to be missing some punctuation, and check capitalization of self

11 and 12: Any more info on these?

[14] here and in many others, style is "pages X-Y", but different style is
used e.g in [7]

[21] publisher missing

[28] cplex capitalization?

The wrong BibTeX style was used. All references have been completed with all available information, DOIs and URLs were added.

Reviewer 3

This paper is about adding constraints in the Constraint Imperative Programming (CIP) style to an OO language, namely, Ruby, and this is called Babelsberg/R. The contribution is mainly about a modern refresh of ideas in Constraint Inmperative Programming mainly Kaleidoscope within a modern OO language and the paper focuses on the implementation in Ruby. I like the thesis of the paper of embedding constraint solvers in a non-solver specific

fashion as this is an old idea which deserves a modern implementation.

We want to point out that Babelsberg is more than a refresh. None of the prior or related work that we are aware of supports using normal, object-oriented abstractions such as classes and OO methods as constraints. We hope this is clearer now in the introduction and related work, especially in the section comparing object-constraint programming to Kaleidoscope.

That said, I found it hard to identify what is the novel research contribution.

Most the paper is written in mostly in an informal way without much details. I wonder how important is the particular OO language, and the specific importance of Ruby. While the authors have picked Ruby, and they probably need to pick a specific language, can't this be done to other OO languages or dynamic languages.

Yes. We had originally mentioned a JavaScript implementation of Babelsberg as future work. We now mention this earlier to make clear these ideas are not tied to Ruby. The reason for choosing Ruby was that one of the authors was already familiar with the VM.

While the authors are focusing on the CIP paradigm, there are also a number of existing constraint OO libraries. The one which comes to mind which has some careful OO integration is Ilog Solver (nowadays called IBM Ilog Solver) which takes specific advantage of C++ making it easy to write semi-declarative programs in C++ including handling of backtracking.
The authors should compare the OO features of Babelsberg/R with Ilog Solver eventhough the CIP focus does make this paper different.

Ilog Solver, like Kaplan or Backtalk, has a different focus, which we now expand upon in the "Related Work" section.

Notice that Ilog Solver does not need to have different execution modes like what is done here because of the use of overloading.

Ilog Solver uses overloading, for example, to be able to do arithmetic with numbers and decision variables. However, Ilog Solver cannot create a constraint expression from expressions involving only ordinary numbers. Instead, the programmer has to explicitly create decision variables to build constraints. These decision variables can then be used (through overloading) in conjunction with normal numbers. However, Babelsberg does not use execution modes in that case either (the VM just reads constrained variables differently and dispatches to the type of their value. So the mechanism is very similar.)

The execution modes are used to avoid having to explicitly create decision variable-like objects. This allows constraints in Babelsberg to use domain specific abstractions (e.g. OO methods) available on the constrained objects, without having to add further overloads to a decision variable class.

Very few constraint solvers are designed for CIP as it requires specific treatment of soft constraints. Its not clear how those solvers, e.g., Z3 will fit well. It would be better to have a clearer treatment of solver semantics combined with language semantics.

While soft constraints are useful for many application domains, CIP does not necessarily require them. The section on "Hard and Soft Constraints" now explains this and also explains that priorities can be interpreted by the solver. In our implementation, Z3 will raise an exception if a priority is passed.

For example, by having a formalization which can encompass several solvers and their different features, this can then be clearly mapped to both syntax, semantics and implementation in Babelsberg/R.

The section "Solving Constraints" now elaborates that solver specifics (such as numeric precision, how they select solutions, or whether they support incremental solving or soft constraints) are not considered part of the language. Instead, programmers choose solvers as they choose among any multiple alternative libraries – by weighing their properties.

The current contribution is not a unification of different solvers' features, but the unification of declaring and solving constraints with object-oriented methods and abstractions. The ability to use ordinary methods in constraints allows programmers to use libraries and domain specific abstractions in constraints without having to express their meaning in both an imperative and a declarative way.

Note that in S2.1, the Z3 example is quite different from the other examples.

This example is not Babelsberg code, but ordinary Ruby code using the Z3 library. We have expanded the text to clarify this.

Because the paper is quite informal, I wonder if there are issues with variables, what happens if an object is no longer live (and hence can be deleted) but the constraints interact with another constraint in another live object. Normally the variables in the dead object can be garbage collected and also the data structures but this may not be correct with constraints. Because there is little treatment of the interaction between language semantics and constraints, examples like this one raises questions about their language integration.
I also wonder if there are other garbage collection issues with constraints.

A description of GC issues is now included in the implementation section. A short explanation: normal variables are garbage collected normally. Constrained variables are referenced from the variable storage locations in the VM, so if those go out of scope, they can be collected, and their last value remains in the solver specific data structures. However, variables can additionally be referenced from the block closures that created the constraints, so additional references may exist. For objects that have no solver representation, the blocks are kept alive (which would be most complex data structures). This means that the variables are kept alive, too

The contribution of the CIP part of the paper is mentioned only briefly.
Is it mainly the implementation?

We hope the extended theory, the shortened implementation section, and the more general design section highlight that the main contributions are not in the implementation, but rather in how Babelsberg uses ordinary OO methods and abstractions in constraints, as well as how solvers can be added to the language.

It is unclear to me in the evaluation section that the imperative version is behaviorally equivalent to the object-constraint version. Shouldn't the constraint version be more general than the imperative version? If so the comparison may not be totally fair for the constraint version.

Yes, it's not totally fair, but in practice we may not care for the more general solution, we just care about making it more concise or expressive. If we assume the different versions came about as part of a refactoring, then we think the comparison is important. We have added this argument to the text.

Overall it seems to me that this paper is interesting but a bit rough and could benefit from more precision and perhaps also generalizability.

Why is Ruby chosen anyway? Why not other OO languages? Why not Java even?

See the answer above.

Overall the motivation of the paper is good but I feel that the paper needs a more careful treatment of the semantics, language issues and implementation.

More specific comments:

S1:
The Fermat's last theorem example is not good. In any case, it has now been proven.

It has been removed.

Rather than say "constraint satisfaction library" - constraint solver would be better to avoid confusions.

Changed to "Constraint Solver Libraries" – the fact that the solver is wrapped in a library for an imperative language is important for our argument. We hope this is also clearer in the updated text now.

S3:
"always { rect.visible? } " might not be very clear at this point.

We have added a reminder to the text of what the method does.

What does "upstream" mean - it is not defined?

This was removed during editing.

As mentioned above, what happens if solver doesn't support soft constraints?

See explanation above.

S4:
As the implementation is probably the main contribution, i think more specifics on how the VM interacts with the solver is needed.

As we do not agree that the implementation is the main contribution, we have instead (as also requested by the other two reviewers) shortened the implementation section and expanded the theory and design to be more generic.

It was not clear what is meant by solvers supporting "if .. then ...".

This has been removed. For an explanation: some solvers, for example Z3, support conditional constraints. If the condition is satisfied, the branch is also satisfied.

The part about avoiding recursions in the solver should be explained better.

An explanatory sentence was added, but the section as a whole is shorter.

For Fig 1, it would be good to also show the constraint.

The constraint is above the caption, it is that the area of the rectangle should be larger or equal to 100.

Are the Edit Constraints tied to the Cassowary solver or can one use other solvers?

We have split the explanation of edit constraints into theory, design, and implementation issues. We hope that this makes it clearer that "edit" is a method in the Cassowary library. This means that the developer has to know about this method and then has to be explicit about using it. The DeltaBlue library also supports edit constraints, but again, the developer has to be aware of this and choose to use DeltaBlue edit constraints explicitly

S5:
It would be good to have a single example which can be run with different solvers and also compared with imperative code (perhaps the imperative code might be big so may need to be in appendix).

The various constraint solvers have quite different capabilities, making it hard to find a single convincing example to use with all of them. Generally, we expect programmers to activate the solvers that support the types they use, and provide the best performance and results. An additional issue with comparing DeltaBlue, Cassowary, and Z3 (for example) is that these are implemented in different languages in Babelsberg/R (Cassowary and DeltaBlue are pure Ruby code, Z3 is a C library that is used through CFFI), and have very different features.

However, we added comparisons between different languages/libraries that support constraint solving (Turtle, Kaplan, Prolog with the CLP library).

General:
The font of language constructs in running text is a bit too small, it doesn't fit well with the font size of text in paragraph mode, e.g. a+b=c on pg 7.

Changed